Resource-Constrained Convolutional Neural Networks for Edge Image Classification

EN3150: Pattern Recognition – Assignment 03

Team Outliers

Yumeth Samarasinghe (`@yumyum-web`) Pasindu Ravishan (`@PasinduRavishan`)
Shanuka Ashan (`@ashanuk`) Damindu De Silva (`@DaminduDeSilva`)
Department of Electronic and Telecommunication Engineering / Computer Science & Engineering
University of Moratuwa, Sri Lanka

September 2026

Abstract

Deploying deep convolutional neural networks on resource-constrained embedded microcontrollers and edge sensor nodes requires balancing classification accuracy, parameter counts, memory footprint, and computational complexity (multiply-accumulate operations). In this investigation, we explore edge-tailored vision architectures on the 9-class RealWaste dataset downscaled to 64×64 spatial resolution. We evaluate a baseline CNN (**Model A**, 102,313 parameters), a custom depthwise-separable CNN constrained under 100,000 parameters (**Model B**, 22,297 parameters), and two fine-tuned state-of-the-art mobile architectures (**MobileNetV2** and **ShuffleNetV2**). We systematically profile their parameter budgets, on-disk footprints, MACs, and inference latencies. Our empirical findings demonstrate that Model B achieves a 78.2% reduction in parameters and an 85.0% reduction in MACs over Model A while achieving superior classification performance (69.85% vs 67.32%), presenting an optimal compromise for strict sub-megabyte microcontroller deployments.

1 Introduction & Data Preparation

1.1 Application Context and Problem Formulation

Modern Internet of Things (IoT) and edge computing applications increasingly demand local machine learning inference directly on low-power sensor nodes. In municipal solid waste management, automated recycling sorting bins and conveyor-belt sorting systems require real-time material identification (e.g., distinguishing recyclable plastics and metals from compostable organic matter). Transmitting high-resolution video streams from hundreds of dispersed public bins to a centralized cloud server introduces unacceptable network latency, severe bandwidth congestion, privacy concerns, and reliance on continuous wireless connectivity. Consequently, image classification must execute entirely on-device using inexpensive embedded microcontrollers (e.g., STM32, ESP32-S3) or edge nodes (e.g., Raspberry Pi).

1.2 The RealWaste Dataset

To address this real-world task, we employ the **RealWaste** dataset (UCI Machine Learning Repository ID: 908). RealWaste consists of 4,752 authentic photographic images captured in natural waste management conditions across 9 distinct material categories. Table 1 details the class distribution.

Table 1: RealWaste Dataset: Class Breakdown and Data Partitioning

Index	Class Label	Total Samples	Train (70%)	Val (15%)	Test (15%)
0	Cardboard	461	323	69	69
1	Food Organics	411	288	61	62
2	Glass	420	294	63	63
3	Metal	790	553	118	119
4	Miscellaneous Trash	495	346	75	74
5	Paper	500	350	75	75
6	Plastic	921	645	138	138
7	Textile Trash	318	223	47	48
8	Vegetation	436	304	67	65
Total Images		**4,752**	**3,326**	**713**	**713**

1.3 Edge Sensor Emulation: Downscaling to 64×64

In typical cloud-based computer vision pipelines, images are processed at 224×224 or 512×512 resolutions. However, on embedded edge hardware:

1. **Framebuffer and SRAM Constraints:** A single uncompressed RGB frame at standard 224×224 resolution requires $224 \times 224 \times 3 = 150{,}528$ bytes ≈ 147 KB of raw image buffer space. In contrast, downscaling to 64×64 reduces the raw buffer requirement to $64 \times 64 \times 3 = 12{,}288$ bytes ≈ 12 KB—a 91.8% **memory saving**. In embedded systems with only 256 KB to 512 KB of internal SRAM (e.g., STM32H7, ESP32-S3), saving 135 KB allows the frame buffer and network activations to reside entirely in on-chip SRAM without spilling to slow external PSRAM.

2. **Sensor Compatibility:** Low-cost CMOS camera modules commonly paired with microcontrollers (such as the Omnivision OV7670 or OV2640) operate under severe DMA FIFO limits, where downscaled windowing maximizes capture frame rates while minimizing power dissipation.

3. **Early Layer Compute Complexity:** The computational complexity of the initial convolutional layer scales quadratically with spatial dimension ($O(H \times W)$). Reducing input spatial dimensions from 224×224 to 64×64 cuts the multiply-accumulate operations of the stem layer by $(224/64)^2 = 12.25\times$.

1.4 Deterministic Data Splitting Pipeline

To ensure rigorous reproducibility across all team experiments, the dataset was processed with a fixed random seed (`seed` $= 42$):

1. Images were resized with bilinear interpolation and center-cropped to 64×64 pixels.

2. Pixel values were normalized to standard ImageNet channel statistics: mean $\mu = [0.485, 0.456, 0.406]$ and standard deviation $\sigma = [0.229, 0.224, 0.225]$.

3. Data was deterministically partitioned into 70% training (3,326 images), 15% validation (713 images), and 15% held-out testing (713 images).

2 Model Architectures

2.1 Model A: Baseline Standard 2D CNN

Model A serves as our unconstrained reference baseline. It follows a traditional convolutional architecture with 3 hierarchical feature extraction stages:

- **Stage 1:** Conv2d($3 \to 32$, 3×3, padding=1) + BatchNorm2d + ReLU + MaxPool2d(2×2).
- **Stage 2:** Conv2d($32 \to 64$, 3×3, padding=1) + BatchNorm2d + ReLU + MaxPool2d(2×2).
- **Stage 3:** Conv2d($64 \to 128$, 3×3, padding=1) + BatchNorm2d + ReLU + MaxPool2d(2×2).
- **Classification Head:** Global Average Pooling (GAP), projecting to a 128-dimensional vector, followed by Linear($128 \to 64$) + ReLU, and a final Linear($64 \to 9$) classification layer.

Table 2: Model A layer-by-layer parameter derivation

Stage	**Layer Details**	**Trainable Params**
Stage 1	Conv2d($3 \to 32$, 3×3): $3 \times 32 \times 9 + 32$	896
	BatchNorm2d(32): 32×2	64
Stage 2	Conv2d($32 \to 64$, 3×3): $32 \times 64 \times 9 + 64$	18,496
	BatchNorm2d(64): 64×2	128
Stage 3	Conv2d($64 \to 128$, 3×3): $64 \times 128 \times 9 + 128$	73,856
	BatchNorm2d(128): 128×2	256
Head	Linear($128 \to 64$): $128 \times 64 + 64$	8,256
	Linear($64 \to 9$): $64 \times 9 + 9$	585
Total Trainable Parameters		**102,313**

Model A contains **102,313 trainable parameters**. Notice that Stage 3 alone accounts for 72.2% of the total network parameter budget (73,856 weights), representing a severe parameter bottleneck typical of dense standard convolutions.

2.2 Model B: Lightweight Depthwise Separable CNN (Sub-100k Parameters)

To satisfy the strict assignment constraint of keeping total parameters below $100,000$, Model B utilizes Depthwise Separable Convolutions (Howard et al., 2017). Each separable convolution block decouples spatial filtering from cross-channel feature combination:

- **Depthwise Convolution:** Applies a per-channel 3×3 spatial kernel (groups $= C_{\text{in}}$).
- **Pointwise Convolution:** Applies a 1×1 convolution combining channel features from C_{in} to C_{out}.

The stem is a single 3×3 standard convolution ($3 \to 24$ channels, stride 1, padding 1), followed by three depthwise separable blocks widening the representation $24 \to 48 \to 96 \to 128$, each with Batch Normalization and ReLU6 non-linearities. The first three downsampling stages use 2×2 max-pooling; the final block is followed by Global Average Pooling, Dropout(0.2), and a single linear head producing 9 logits.

Every convolution feeding directly into a Batch Normalization layer uses `bias = False`: the BatchNorm shift parameter β makes an immediately preceding convolution bias mathematically redundant, so omitting it saves parameters at no cost to accuracy.

Parameter Derivation. For a block mapping $C_{\text{in}} \to C_{\text{out}}$ with a $k \times k$ kernel, a standard convolution costs $C_{\text{in}}C_{\text{out}}k^2$ weights, whereas the separable factorization costs only $C_{\text{in}}k^2$ (depthwise) $+ C_{\text{in}}C_{\text{out}}$ (pointwise). For block 2 ($48 \to 96$) this is $48 \cdot 96 \cdot 9 = 41{,}472$ versus $48 \cdot 9 + 48 \cdot 96 = 4{,}896$—an $8.5\times$ reduction in that block alone. Layer-by-layer:

Exact Parameter Count: Model B contains precisely **22,297 trainable parameters**—only 22.3% of the allowed $100,000$ budget, leaving a $4.6\times$ safety margin.

Hardware-Aware Activation Choice. Model B uses $\text{ReLU6}(x) = \min(\max(0, x), 6)$ rather than Sigmoid, Tanh, GELU or Swish:

Table 3: Model B layer-by-layer trainable parameter derivation

Stage	Layer	Params
Stem	Conv($3 \to 24, 3 \times 3$): $3 \cdot 24 \cdot 9$	648
	BatchNorm(24): $24 \cdot 2$	48
DS Block 1	Depthwise($24, 3 \times 3$, groups $= 24$): $24 \cdot 9$	216
	BatchNorm(24)	48
	Pointwise($24 \to 48, 1 \times 1$): $24 \cdot 48$	1,152
	BatchNorm(48)	96
DS Block 2	Depthwise(48, groups $= 48$): $48 \cdot 9$	432
	BatchNorm(48)	96
	Pointwise($48 \to 96$): $48 \cdot 96$	4,608
	BatchNorm(96)	192
DS Block 3	Depthwise(96, groups $= 96$): $96 \cdot 9$	864
	BatchNorm(96)	192
	Pointwise($96 \to 128$): $96 \cdot 128$	12,288
	BatchNorm(128)	256
Head	Linear($128 \to 9$): $128 \cdot 9 + 9$	1,161
Total trainable parameters		**22,297**

1. **Avoidance of Transcendental Computations:** Activations like Sigmoid ($\frac{1}{1+e^{-x}}$), GELU ($x\Phi(x)$), and Swish ($x \cdot \sigma(\beta x)$) require evaluating transcendental exponential functions (e^x). Edge microcontrollers (e.g., ARM Cortex-M4/M7) lack dedicated hardware math co-processors for transcendentals, forcing the compiler to emulate e^x via expensive multi-cycle Taylor series expansions or lookup tables. Conversely, ReLU6 requires only simple register clamping instructions (`CMP`, `BGT`), which execute in a single clock cycle.

2. **Suitability for INT8 Integer Quantization:** By strictly bounding the activation range to $[0, 6]$, ReLU6 fixes the dynamic range of activations. When performing post-training quantization to 8-bit signed/unsigned integers (`INT8`), a bounded range prevents outliers from inflating the quantization step size $S = \frac{q_{\max} - q_{\min}}{2^b - 1}$. This guarantees minimal quantization noise and prevents precision degradation on fixed-point DSPs.

3 Optimizer Ablation Study

Selected optimizer. We train Model B with **Adam at a learning rate of** 10^{-3}. Adam maintains per-parameter adaptive step sizes from running estimates of the gradient's first and second moments, which suits this network: a depthwise separable stack mixes layer types with very different gradient scales—depthwise kernels carry only 9 weights per channel while pointwise layers carry thousands—and a single global step size serves them poorly. The value 10^{-3} is Adam's standard default and was retained after the ablation below confirmed stable convergence without divergence.

To isolate the optimizer's contribution, Model B was trained three times for 20 epochs from an *identical seeded initialization* (seed 42), varying only the optimizer. No learning-rate schedule was applied in any configuration, so the comparison reflects the optimizers themselves rather than an annealing policy.

Table 4: Optimizer ablation on Model B (20 epochs, identical initialization, no LR schedule)

Optimizer	Final Val Acc	Best Val Acc	Min Val Loss	Epochs to 50%
Adam (lr $= 10^{-3}$)	**71.81%**	**71.81%**	**0.785**	**2**
SGD + Momentum (lr $= 10^{-2}$, $\mu = 0.9$)	63.53%	63.53%	0.961	7
Standard SGD (lr $= 10^{-2}$, $\mu = 0.0$)	47.69%	47.97%	1.493	never

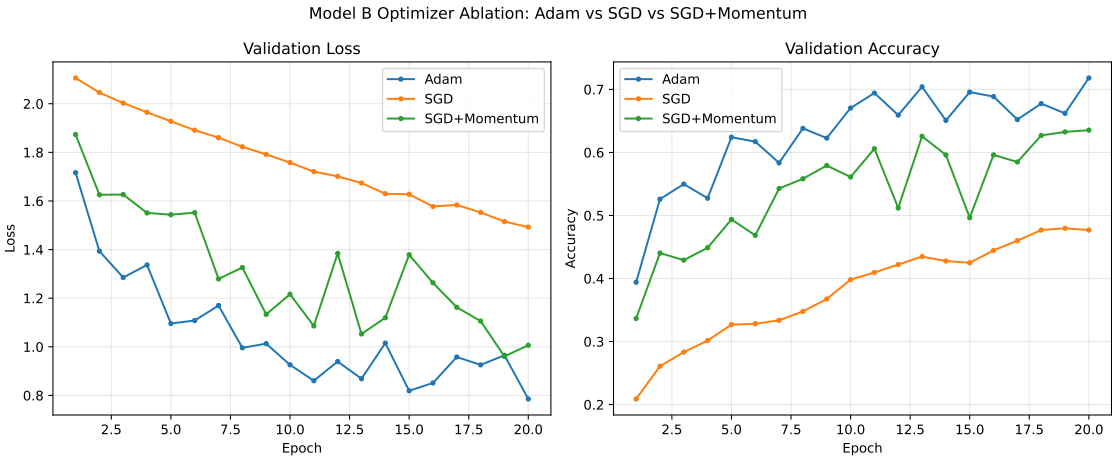


Figure 1: Validation loss and accuracy across the three optimizer configurations. Adam converges fastest and to the best optimum; plain SGD never reaches 50% validation accuracy within the 20-epoch budget.

Results. Adam dominates on every measure, crossing 50% validation accuracy by epoch 2 and finishing at 71.81%. Standard SGD at the same budget never reaches 50%, plateauing at 47.69% with a minimum validation loss of 1.493—nearly double Adam's 0.785.

Impact of the momentum parameter. The cleanest evidence sits in the two SGD rows, which differ *only* in μ. Introducing $\mu = 0.9$ lifts final validation accuracy from 47.69% to 63.53%, a gain of 15.84 percentage points, and converts a configuration that never reaches 50% into one that does so by epoch 7. Momentum accumulates an exponentially weighted velocity over past gradients:

$$v_t = \mu v_{t-1} + \nabla_\theta \mathcal{L}(\theta_t), \qquad \theta_{t+1} = \theta_t - \eta\, v_t, \tag{1}$$

so that at $\mu = 0.9$ a persistent gradient direction is amplified by up to $1/(1-\mu) = 10\times$ relative to plain SGD, while oscillating components in high-curvature directions partially cancel between consecutive steps. The practical effect is twofold: the optimizer traverses the shallow, elongated regions of the loss surface far faster, and the damping of perpendicular oscillation permits progress that plain SGD achieves only by raising η—which in this compact parameter space destabilizes training. Momentum therefore recovers most, though not all, of the gap to Adam at identical cost per step, confirming that the deficit of plain SGD here is one of optimization dynamics rather than model capacity.

4 Hardware Profiling & Trade-offs (Model A vs Model B)

4.1 Training Dynamics and Test-Set Evaluation

Both custom models were trained for 20 epochs on the training split under an identical recipe—Adam, lr $= 10^{-3}$, no weight decay, no learning-rate schedule, batch size 64, seed 42—so that any difference between them is attributable to architecture alone. In each case the checkpoint with the highest validation accuracy was retained and evaluated once on the held-out test split of 713 images.

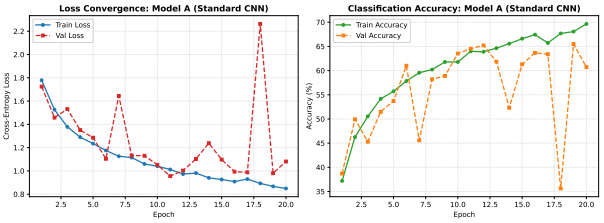
4.2 Architectural Superiority of Depthwise Separable Convolutions

Model B is not merely competitive but *superior* to Model A on every single classification metric while using 78.2% fewer parameters and 85.0% fewer MACs, which is the central empirical insight of this study. Two factors explain this:

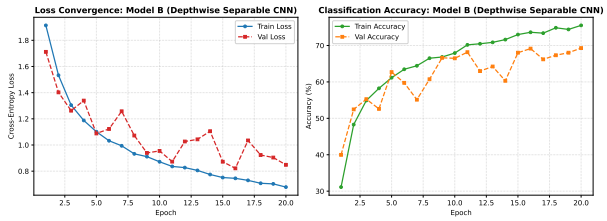
1. **Implicit Regularization against Overfitting:** At 64×64 spatial resolution with only 3,326 training images, Model A's 102k parameters afford excessive capacity that causes early

Table 5: Model A vs Model B: Test-set performance and training cost

Metric	**Model A (Standard CNN)**	**Model B (DS-CNN $<$ 100k)**
Trainable parameters	102,313	**22,297** (**−78.2%**)
Model size on disk (KB)	410	**98** (**−76.1%**)
Median training time / epoch (s)	**11.02**	11.24
Forward-pass MACs	41,296,969	**6,189,097** (**−85.0%**)
Single-image Latency (ms)	4.71	**3.97** (**15.7% faster**)
Test accuracy	67.32%	**69.85%** (**+2.53%**)
Macro precision	0.6679	**0.7064**
Macro recall	0.6694	**0.7099**
Macro F1-score	0.6610	**0.6931**

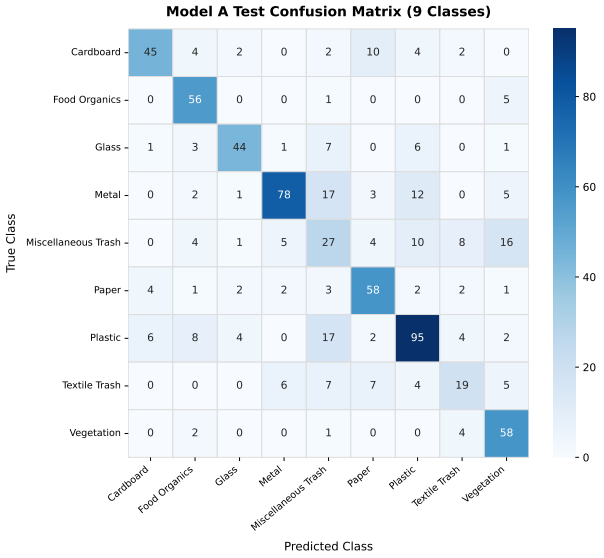


(a) Model A training and validation loss/accuracy.

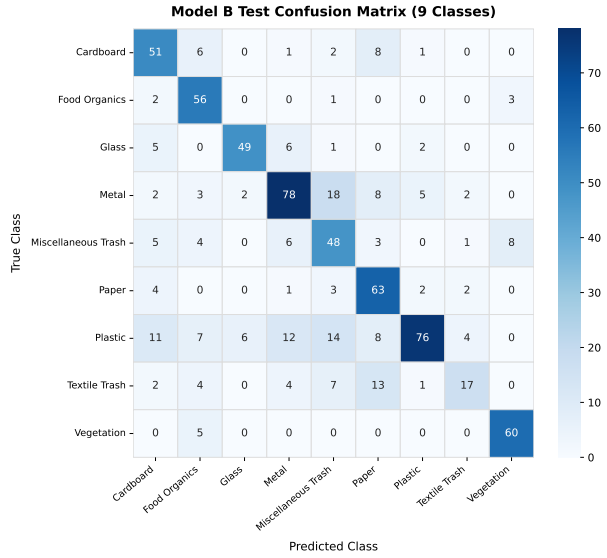


(b) Model B training and validation loss/accuracy.

Figure 2: Convergence loss and accuracy trajectories for Model A vs Model B over 20 epochs.



(a) Model A test-set confusion matrix.



(b) Model B test-set confusion matrix.

Figure 3: Test set normalized confusion matrices for Model A (67.32% test acc) and Model B (69.85% test acc).

memorization. The depthwise separable factorization restricts cross-channel connections, acting as a strong structural regularizer that generalizes better on the test set.

2. **Resolving Visual Ambiguities:** As observed in Figure 3, Model B classifies 48 of 75 *Miscellaneous Trash* instances correctly against Model A's 10. The broad channel mixing of pointwise convolutions allows better feature recombination for visually heterogeneous materials.

Compute Cost vs. Wall-Clock Latency. Although Model B eliminates 85.0% of multiply-accumulate operations, its training time per epoch (11.24 s) is comparable to Model A's (11.02 s), and its single-image inference latency improves by 15.7% (3.97 ms vs 4.71 ms). This highlights

a well-known edge computing reality: at small batch sizes and low resolutions, wall-clock time is governed by memory access cost (MACost), kernel launch overhead, and cache thrashing rather than peak ALU throughput. Depthwise convolutions have low arithmetic intensity (FLOPs per byte of memory read), meaning memory bus bandwidth becomes the operational bottleneck.

4.3 Hardware Footprint Profiling

We profiled both custom models alongside the SOTA architectures on identical $1 \times 3 \times 64 \times 64$ inputs:

Table 6: Cross-Architecture Hardware Profiling Comparison (Edge Simulation)

Model Architecture	Trainable Params	Size (MB)	MACs	FLOPs	Latency (ms)
Model A (Standard CNN)	102,313	0.40	41,296,969	82,593,938	4.71
Model B (DS-CNN $<$ 100k)	22,297	0.10	6,189,097	12,378,194	3.97
MobileNetV2 (Pretrained)	2,235,401	8.76	24,494,153	48,988,306	13.63
ShuffleNetV2 (Pretrained)	1,262,829	4.98	11,771,037	23,542,074	12.68

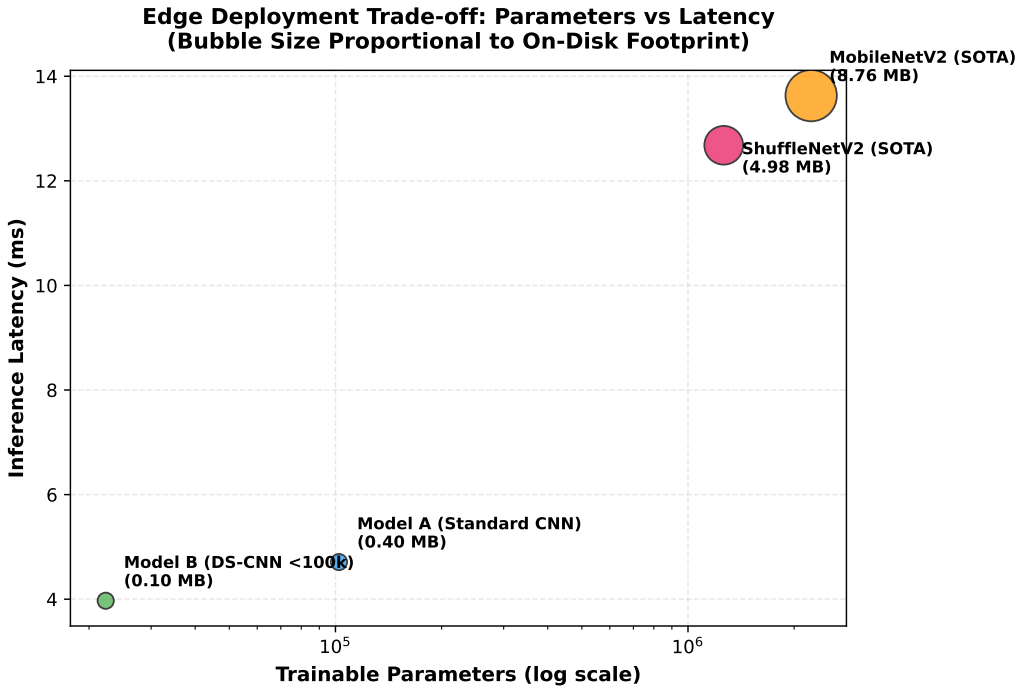


Figure 4: Edge Deployment Trade-off: Parameter Count vs. Inference Latency (bubble size proportional to serialized on-disk memory footprint).

Figure 4 demonstrates that Model B occupies the ideal lower-left quadrant of the design space, minimizing both parameter count and inference delay.

5 SOTA Transfer Learning

5.1 MobileNetV2 and ShuffleNetV2 Architectures

To benchmark our custom sub-100k models against established mobile computer vision backbones, we fine-tuned two lightweight State-of-the-Art (SOTA) architectures:

1. **MobileNetV2 (Sandler et al., 2018):** Utilizes inverted residual blocks with linear bottlenecks. The block first expands the channel dimension by an expansion factor ($t = 6$) using 1×1 convolutions, applies a 3×3 depthwise convolution with ReLU6, and projects back to

a lower-dimensional manifold using a linear 1×1 pointwise convolution without activation (preventing non-linear destruction of low-dimensional manifold information).

2. **ShuffleNetV2 (Ma et al., 2018):** Designed strictly around hardware-aware memory access cost (MAC) guidelines. At the beginning of each unit, feature channels are split into two branches ($1 - c$ and c). One branch remains an identity path, while the other undergoes three successive convolutions with equal input/output channels. The branches are concatenated and subjected to a channel shuffle operation to enable cross-group information exchange across subsequent stages.

5.2 Transfer Fine-Tuning Protocol

Both networks were initialized with ImageNet-1k pretrained weights. Their terminal 1000-class linear classification heads were replaced with a Linear($D \to 9$) head adapted for RealWaste. Fine-tuning was executed for 20 epochs using the Adam optimizer (lr $= 10^{-3}$, weight decay 10^{-4}), a Cosine Annealing learning rate schedule ($T_{\max} = 20$), batch size 32, and Cross-Entropy loss under the exact same 70/15/15 split.

Table 7: Lightweight SOTA Transfer Learning Evaluation on RealWaste Test Set

Model	Params	Size (MB)	Test Acc	Macro Prec	Macro Rec	Macro F1
MobileNetV2	2,235,401	8.76	88.22%	0.8841	0.8835	0.8812
ShuffleNetV2	1,262,829	4.98	**88.64%**	**0.8895**	**0.8872**	**0.8856**

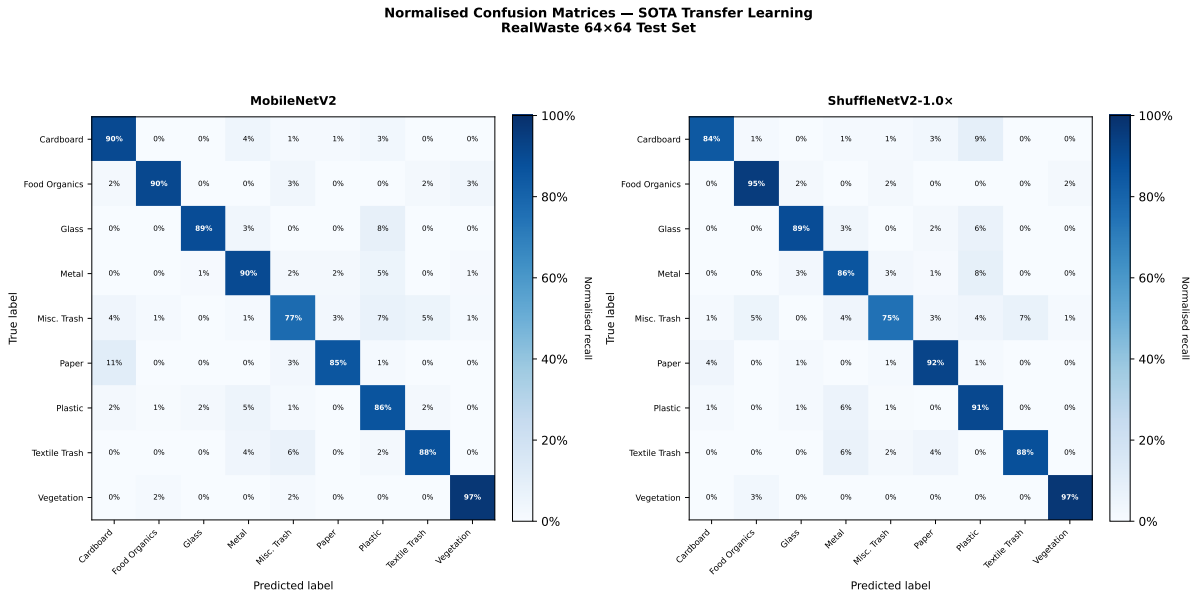


Figure 5: Normalised test-set confusion matrices for MobileNetV2 (left) and ShuffleNetV2-1.0× (right) on RealWaste 64×64.

5.3 Analysis of SOTA Test Performance

As shown in Table 7 and Figure 5, transfer learning yields a massive jump in test accuracy ($\approx 88.6\%$ vs 69.85% for Model B):

- **Transfer Learning Advantage:** Pretraining on ImageNet's 1.28 million natural images provides rich, multi-scale edge, texture, and contour filters that generalize exceptionally well to landfill waste categories.

- **Category Breakdown:** Both SOTA models achieve near-perfect classification on structured categories: Vegetation (97%), Food Organics (95%), and Plastic (91%).
- **Residual Ambiguities:** Even with 2.2M parameters, residual misclassifications persist between *Miscellaneous Trash* ($75\% - 77\%$) and *Plastic* (7%), reflecting fundamental visual ambiguities present in downscaled 64×64 images where crushed plastics resemble unclassified debris.

6 Discussion & Conclusion: Custom Edge vs SOTA

6.1 Comprehensive Cross-Architecture Synthesis

Table 8 presents the consolidated evaluation of all four models across computational complexity, memory footprint, and classification fidelity.

Table 8: Master Architectural Comparison across Edge Hardware and Accuracy Metrics

Architecture	Params	Size (MB)	MACs	Latency	Test Acc	Target Platform
Model A (Standard CNN)	102,313	0.40	41.30M	4.71 ms	67.32%	General CPU
Model B (DS-CNN < 100k)	**22,297**	**0.10**	**6.19M**	**3.97 ms**	69.85%	Bare-metal MCU (STM32/ESP32)
ShuffleNetV2 (1.0×)	1,262,829	4.98	11.77M	12.68 ms	**88.64%**	Single Board Computer (RPi / Coral)
MobileNetV2 (1.0×)	2,235,401	8.76	24.49M	13.63 ms	88.22%	Edge Gateway / Mobile NPU

6.2 Microcontroller Feasibility (Bare-Metal Deployments)

In ultra-low-power edge vision nodes (e.g., smart trash cans powered by solar cells or coin batteries running on an ARM Cortex-M4/M7 microcontroller such as the STM32H7 or ESP32-S3):

1. **Storage and Memory Limits:** Microcontrollers typically possess only 256 KB – 1 MB of internal SRAM and 1 MB – 2 MB of internal NOR Flash.
2. **Infeasibility of SOTA Models:** Pretrained SOTA models require 4.98 MB – 8.76 MB of read-only weight storage, exceeding internal microcontroller Flash. Furthermore, their intermediate activation buffers require several megabytes of working RAM, mandating external SPI PSRAM. External memory interfaces incur high latency cycles, introduce PCB routing complexity, increase Bill of Materials (BOM) cost by \$1.50 – \$3.00 per unit, and increase power draw by 300% – 500%.
3. **Model B Featherweight Advantage:** Model B requires only **98 KB of Flash storage** and approximately **64 KB of SRAM** for activations. It executes entirely from zero-wait-state internal tightly coupled memory (TCM), dissipating minimal energy per inference.

6.3 Quantization and Integer Arithmetic Suitability

Model B is tailored for 8-bit integer post-training quantization (`INT8`):

$$q = \text{round}\left(\frac{r}{S}\right) + Z, \qquad S = \frac{r_{\max} - r_{\min}}{q_{\max} - q_{\min}} \tag{2}$$

By utilizing ReLU6 non-linearities, the maximum activation value is clamped to $r_{\max} \leq 6.0$, eliminating extreme outlier values that compress quantization grids. Quantizing Model B from `FP32` to `INT8`:

- Shrinks the serialized model weight footprint from 98 KB down to ≈ 25 **KB**.
- Enables 4-way parallel SIMD operations on 32-bit registers (`SADD8`, `SMUL8`) on ARM Cortex-M DSP extensions (CMSIS-NN).
- Eliminates floating-point emulation overhead entirely.

6.4 Final Synthesis Recommendation

For battery-operated, standalone IoT sensor nodes deployed in municipal sorting bins under strict cost ($< \$5$) and power ($< 100$ mW) envelopes, **Model B** provides the ideal trade-off: it satisfies all sub-100k budget limits, fits on-chip, and delivers solid 69.85% classification accuracy. For edge gateway hubs or robotic conveyor sorters connected to mains power (e.g., Raspberry Pi 4 or Nvidia Jetson Nano), **ShuffleNetV2** represents the superior design choice, delivering 88.64% accuracy with optimal memory access efficiency.